

# EVOLUTION AI

下一代 AI 汽车造型开发平台 — 从一句话到 3D 整车，全链路 AI 辅助设计

status **v1.02-stable** version **1.2.0** frontend **Vue 3 + Vite + Three.js** backend **FastAPI**

## 🚀 项目愿景

让汽车造型设计进入「**说一句话，就出 3D 整车**」的时代。  
从概念草图 → 参数化建模 → AI 优化 → 视频分镜 → 工程交付，一气呵成。

## 📦 当前状态 (v1.2, 2026-09-27 全面复盘)

经全面复盘对齐现状，三层测试全绿。

| 模块            | 状态        | 技术栈                                              | 规模 (实测)                                                                           |
|---------------|-----------|--------------------------------------------------|-----------------------------------------------------------------------------------|
| 前端            | ✅ 可运行     | Vue 3 + Vite 5 + Three.js + Element Plus + Pinia | <b>10 个路由页面</b> ，3D 实时渲染，i18n 双语                                                  |
| 后端            | ✅ 可运行     | FastAPI + SQLAlchemy 2.0                         | <b>16 个路由模块 / 118 个端点 / 11 张 ORM 表</b>                                            |
| 算法层           | ✅ 可运行     | Python 3.11+ / NumPy / SciPy / Trimesh           | NURBS 整车 + freeform + surface_quality + storyboard; <b>200 pytest + 自检 5 模块全过</b> |
| 测试基线          | ✅ 489+ 全绿 | algorithm_model 200 / backend 178 / frontend 111 | 详见 <a href="#">架构文档</a>                                                           |
| Mock Fallback | ✅ 已集成     | -                                                | backend 不可用时显示 mock 项目，页面不黑屏                                                      |
| 可选能力          | ✅ 真实降级    | PyTorch / Ollama / Redis / CLIP                  | 缺失时 503 或自动降级，绝不伪装成功                                                              |
| 部署            | ✅ Docker  | Compose + Nginx + cpolar 隧道                      | 见 <a href="#">deploy/</a>                                                         |

## 🚀 快速开始

唯一活跃开发目录: **D:\API\Evolution-Ai.Design**

## 1. 启动前端 (Vite Dev Server, 端口 5173)

```
cd D:\API\Evolution-Ai.Design
npm install      # 首次运行
npm run dev      # 启动开发服务器
# 访问 http://localhost:5173/
```

路由使用 Hash 模式 (适配 GitHub Pages 静态部署) :

- 首页: <http://localhost:5173/#/>
- AI 设计器: <http://localhost:5173/#/designer>
- 项目列表: <http://localhost:5173/#/projects>
- 项目详情: <http://localhost:5173/#/projects/1>

## 2. 启动后端 (FastAPI, 端口 8000, 可选)

前端已内置 mock fallback, 后端可选。如需完整功能:

```
cd D:\API\Evolution-Ai.Design\backend
pip install -r requirements.txt

# Redis 可选 (会话持久化增强; 未安装时自动降级到内存)
# redis-server --port 6379

# 起 FastAPI
.\start_backend.bat
# 或: python start.py
# API 文档: http://localhost:8000/docs
```

一键启动/重启三服务 (前端、后端、cpolar 隧道) 可用

[.trae/skills/launch-services-detached](#) 独立进程脚本, 详见该技能说明。

## 3. 构建生产版本

```
npm run build      # 输出到 dist/
npm run preview     # 本地预览生产构建
```

## 4. 跑测试

```
# 前端测试
npm test

# 后端测试
cd backend
pytest tests/ -v

# 算法层自检
cd algorithm_model
python test_all.py
```

## 架构设计

详见: [docs/ARCHITECTURE\\_DESIGN.md](#)

### 核心架构 5 层分层:

```
L1 前端层      - Vue 3 + Three.js + Element Plus + Pinia
L2 API 网关    - Vite Dev Proxy (/api/v1 → 8000, /api/ide → trae-api-cn)
L3 后端服务    - FastAPI + 16 个薄壳路由模块 (118 端点, 编排不重写算法)
L4 算法层      - algorithm_model (独立包; 7 大高层 API, 黑盒使用)
L5 基础设施    - SQLite + 文件存储; 可选 Redis
```

### 关键设计决策:

- 1. ☒ 算法模型与 Web 完全解耦 (可独立 pip install)
- 2. ☒ 后端只做编排, 不重写算法
- 3. ☒ 长任务后台化 (训练等耗时任务在后台线程执行, 接口立即返回任务对象, 客户端轮询状态)
- 4. ☒ 前端内置 mock fallback (backend 不可用时不黑屏)

## 📁 项目结构

Evolution-Ai.Design/

|                           |                                                  |
|---------------------------|--------------------------------------------------|
| — src/                    | ← Vue 3 前端源码                                     |
| — views/                  | ← 10 个路由页面                                       |
| — Dashboard.vue           | ← 首页总览                                           |
| — Designer.vue            | ← AI 设计器 (参数+3D预览)                               |
| — Projects.vue            | ← 项目列表 (含 mock fallback)                         |
| — ProjectDetail.vue       | ← 项目详情                                           |
| — DeepLearning.vue        | ← 深度学习设计器                                        |
| — Quality.vue             | ← 质量检查                                           |
| — Deliver.vue             | ← 工程交付 (导入改参导出)                                  |
| — Login.vue / Account.vue | ← 登录 / 账户                                        |
| — Demo.vue                | ← DEMO 演示                                        |
| — components/             | ← 复用组件 (Car2D/Car3D/Markdown/TechMatrix)         |
| — stores/                 | ← Pinia 状态 (auth/designer/project/ui)            |
| — config/                 | ← carPresets 品牌车型预设                              |
| — data/                   | ← mock 数据 + 配置                                   |
| — utils/                  | ← carImageManager / imageGenerator / llm         |
| — api.js                  | ← axios 实例 + API 模块 (含 bayesAPI)                 |
| — router.js               | ← vue-router (Hash 模式 + 登录守卫)                    |
| — i18n.js                 | ← vue-i18n 双语                                    |
| — main.js                 | ← 应用入口                                           |
| — backend/                | ← FastAPI 后端                                     |
| — app/                    |                                                  |
| — routes/                 | ← 16 个路由模块 (118 端点)                              |
| — bayes_optimizer.py      | ← 贝叶斯优化引擎 (GP+EI/UCB)                            |
| — car_generator.py        | ← 车身生成器                                          |
| — brand_knowledge.py      | ← 品牌知识库查询                                        |
| — cad_importer.py         | ← CAD 导入与降级                                      |
| — database.py             | ← SQLAlchemy 2.0 ORM (11 表)                      |
| — main.py                 | ← FastAPI 入口                                     |
| — config/                 | ← automotive_parameters / brand_design_knowledge |
| — tests/                  | ← pytest 178 测试                                  |
| — requirements.txt        |                                                  |
| — algorithm_model/        | ← 算法核心 (独立可安装)                                   |
| — car_modeling/           | ← 参数化整车 (body/body_nurbs/assembler/...)          |
| — surface_quality/        | ← 曲面质量 (G0/G1/G2 + 反射线 + 优化)                     |
| — freeform/               | ← NURBS 核心 + 扫掠 + 圆角 + STEP writer               |

|                              |                                          |
|------------------------------|------------------------------------------|
| storyboard/                  | ← 分镜生成 + viewer                          |
| examples/                    | ← 17 个示例                                 |
| tests/                       | ← pytest 200 测试                          |
| deploy/                      | ← Docker Compose / Dockerfile / Nginx 配置 |
| docs/                        | ← 文档（架构/产品/API/贝叶斯/审计报告）                 |
| scripts/                     | ← 历史数据采集与训练辅助脚本                          |
| tests/                       | ← 前端 vitest 测试                           |
| rhino/、backend/rhino_plugin/ | ← Rhino 品牌 DNA 面板与插件                     |
| vite.config.js               | ← Vite 配置（端口 5173，代理）                    |
| package.json                 | ← npm 依赖                                 |
| index.html                   | ← HTML 入口                                |

## 🔧 技术栈

| 层   | 技术                                                                                                              |
|-----|-----------------------------------------------------------------------------------------------------------------|
| 前端  | Vue 3.4 / Vite 5.3 / Three.js 0.166 / Element Plus 2.7 / Pinia 2.1 / vue-router 4.4 / vue-i18n 9.13 / axios 1.7 |
| 后端  | FastAPI / Uvicorn / Pydantic v2 / SQLAlchemy 2.0 / PyJWT / cryptography / httpx; 可选 Redis                       |
| 算法层 | Python 3.11+ / NumPy / SciPy / Trimesh / Pillow / Cython 加速                                                     |
| 数据库 | SQLite (开发/当前); PostgreSQL (规划)                                                                                 |
| 部署  | Docker Compose + Nginx (已实现) / cpolar HTTPS 隧道 / GitHub Pages (静态)                                              |

## 🎯 路线图进展 (NURBS+STEP 工程化)

双轨并行策略已落地：

|                                                 |                                           |                                                |                                          |
|-------------------------------------------------|-------------------------------------------|------------------------------------------------|------------------------------------------|
| STEP writer <input checked="" type="checkbox"/> | 单曲面闭环 <input checked="" type="checkbox"/> | 车身 NURBS 化 <input checked="" type="checkbox"/> | 全车装配 <input checked="" type="checkbox"/> |
| freeform/step_writer                            | SweptSurface 已验证                          | body_nurbs + body_ends(G1)                     | 17 个 examples SOP 报告 / FreeCAD 宏         |

当前双管线策略：

- mesh 管线（trimesh 参数化网格）→ 前端实时预览、STL/GLB 导出
- NURBS 管线（自研 NURBS 核心 + 纯 Python STEP writer）→ STEP/IGES 工程输出、A 级曲面 G0/G1/G2 分析

- 参数直驱 NURBS，避免 mesh→NURBS 拟合误差
- 下一阶段：贝叶斯优化与 NURBS 目标函数深度联动、扩展工程约束自动检查（详见 [架构文档](#)）

## 性能基线

| 场景                 | 耗时    | 指标         |
|--------------------|-------|------------|
| 算法层 pytest (200 例) | ~3.8s | 200/200 通过 |
| 算法层一站式自检 (5 模块)    | ~7.2s | 全部通过       |
| 后端 pytest (178 例)  | ~34s  | 178/178 通过 |
| 前端 vitest (111 例)  | ~2.5s | 111/111 通过 |

## 3 套预设方案

| 方案     | L (m) | W (m) | H (m) | 风格   |
|--------|-------|-------|-------|------|
| sport  | 4.50  | 1.85  | 1.30  | 跑车   |
| luxury | 5.20  | 1.95  | 1.50  | 豪华轿车 |
| suv    | 4.80  | 1.95  | 1.72  | SUV  |

## 文档导航

- [架构设计文档](#) — 五层分层、模块清单、数据流、部署（必读）
- [技术白皮书](#) — 平台定位、核心能力、架构部署、可信基线与演进
- [产品功能定义](#) — 用户角色、10 页面功能、能力矩阵
- [API 参考总览](#) — 16 模块 118 端点明细与认证标记
- [英文学术论文](#) — 贝叶斯优化 + NURBS A 级曲面 + DL/LLM 融合 + 端到端集成
- [造型开发方法论](#) — 五维度方法论、SOP、成熟度自检
- [汽车造型设计哲学](#) — 形·理·数，观察·约束·生成·演化，跨文明哲学根基
- [设计元理论与自主知识体系](#) — Love 九层框架、十维度知识体系、中国造型哲学（考工记/谢赫六法）
- [贝叶斯优化模块使用文档](#) — GP + EI/UCB 代理寻优容器，与训练模块联动
- [平台验证报告 2026-09-27](#) — 489 全绿、收敛与事实核对（最新）
- [审计报告 2026-08-10](#) — 历史审计基线
- [平台测试报告 2026-08-11](#) — 历史平台测试记录
- [算法模型文档](#) — 7 大 API + CLI 速查

---

## 开发规范

---

- **算法层业务零修改**: `algorithm_model/` 是独立黑盒, service 层只调不写; 修复契约缺陷 (如自检漂移) 时保持接口向后兼容
  - **后端服务薄壳**: 每个 service 10-50 行, 纯调度
  - **字段 100% 对齐**: Pydantic model 与 `algorithm_model` 数据类签名必须严格一致
  - **测试驱动**: 新功能必须配测试用例
  - **长任务约定**: 训练等耗时任务在后台线程执行, 接口返回任务对象, 客户端轮询 `GET /api/v1/ai/tasks/{id}`
  - **Mock Fallback 优先**: 前端关键页面必须有 mock 数据兜底, backend 不可用时不黑屏
  - **Hash 路由**: 使用 `createWebHashHistory` 适配 GitHub Pages 静态部署
- 

## 协作约定

---

- **唯一活跃目录**: `D:\API\Evolution-Ai.Design` (`D:\API\EVOLUTION_AI` 已退役)
  - **外部归档**: `D:\API\_archive\` (历史快照与 tar.gz)
  - **沟通风格**: 先结论后依据; 少解释过程
  - **文件引用**: 用绝对路径
  - **复盘节奏**: 阶段复盘沉淀到 `docs/`
  - **版本发布**: 成熟版本及时更新 GitHub 仓库, 让网站早日恢复常态化运行
- 

## License

---

Proprietary — 内部项目

---

「说一句话, 就出 3D 整车」 — EVOLUTION AI